

REVISÃO B1 – SISTEMA DISTRIBUIDOS

1. O que é um sistema distribuído?

É quando **vários computadores estão conectados e trabalham juntos**, dividindo tarefas.

Obs: Mesmo tendo várias máquinas por trás, para o usuário pode parecer que está usando **um único sistema**.

Por que usar vários computadores?

Escalabilidade: permite aumentar a capacidade do sistema para atender mais usuários.

Disponibilidade: ajuda a manter o sistema disponível, mesmo quando algum servidor apresenta falha.

Tolerância a falhas: permite que o sistema continue funcionando mesmo quando uma parte dele falha.

Características dos Sistemas Distribuídos:

- **Compartilhamento de recursos:** máquinas compartilham dados e serviços.
- **Concorrência:** várias tarefas podem acontecer ao mesmo tempo.
- **Relógios independentes:** cada máquina possui seu próprio relógio.
- **Falhas independentes:** uma máquina pode falhar sem que todas falhem.

2. Cliente-Servidor × P2P

Cliente-Servidor: o cliente solicita um serviço e o servidor fornece/responde.

P2P: os computadores se comunicam diretamente e podem receber recursos entre si.

Guarda essa diferença:

Cliente-Servidor → cliente **PEDE**, servidor **SERVE**.

P2P → os dois podem **PEDIR** e **SERVIR**.

3. **Desafios**

1. Sincronização → **COORDENAR**

Fazer as máquinas e tarefas trabalharem de forma coordenada.

2. Concorrência → **CONTROLAR**

Controlar várias tarefas acontecendo ou acessando recursos ao mesmo tempo.

3. Falhas → **RESOLVER**

Detectar e tratar problemas quando uma máquina ou comunicação falha.

4. Segurança → **PROTEGER**

Proteger os dados e a comunicação entre as máquinas.

5. Comunicação/Rede → **COMUNICAR**

Garantir a troca de informações, mesmo com atrasos ou problemas na rede.

4. **Arquiteturas de Sistemas Distribuídos**

Arquitetura em Camadas: divide o sistema em camadas, e **cada camada possui uma função específica**.

 **Camadas = separar funções.**

Arquitetura em Nuvem: utiliza **recursos e serviços disponíveis pela rede**, sem depender apenas de uma máquina local.

 **Nuvem = recursos pela rede.**

Arquitetura baseada em Objetos: o sistema é formado por **objetos** que possuem dados e comportamentos e interagem entre si.

🧠 **Objetos = interagir.**

Arquitetura baseada em Eventos: quando um acontecimento gera uma reação no sistema.

🧠 **Eventos = aconteceu → reagiu.**

5. **Processos**

Processo: é um programa que está sendo executado. Possui seu próprio espaço de memória.

Exemplo: Chrome aberto e rodando = processo.

Processo = programa rodando + memória própria.

6. **Threads**

Thread: é uma **tarefa sendo executada dentro de um processo** e compartilha a memória do processo com outras threads.

Exemplo: No Chrome, um thread pode **carregar uma página** enquanto outra **reproduz um vídeo**.

⚠️ **Diferença importante Processo x Threads**

Processos → memória separada.

Threads do mesmo processo → memória compartilhada.

8. Comunicação entre Processos (IPC)

IPC: é a forma usada para **dois ou mais processos se comunicarem e trocarem informações**.

Tipos de IPC

Troca de mensagens: os processos **mandam informações um para o outro**.

Memória compartilhada: os processos **usam o mesmo espaço para trocar informações**.

9. TCP × UDP

TCP: prioriza a **entrega correta e na ordem** dos dados. É mais confiável, mas tem mais controle na comunicação.

 **TCP = CONFIANÇA.**

UDP: envia os dados **sem garantir que todos chegarão ou chegarão na ordem**. Tem menos controle e menor sobrecarga.

 **UDP = RAPIDEZ.**

10. RPC (Remote Procedure Call)

RPC: permite que um programa **execute/chame uma função que está em outro computador pela rede**.

Macete: **RPC = chamar função de longe**.

11. Comunicação Síncrona × Assíncrona

Síncrona: envia uma solicitação e **espera a resposta** para continuar.

🧠 **Síncrona = ESPERA.**

Assíncrona: envia uma solicitação e **continua fazendo outras tarefas**, sem precisar esperar a resposta naquele momento.

🧠 **Assíncrona = NÃO ESPERA.**

12. **Concorrência e Condição de Corrida**

Concorrência: quando **várias tarefas estão acontecendo e disputando/compartilhando recursos** em um mesmo período.

Exemplo: duas pessoas tentando comprar o **último ingresso** quase ao mesmo tempo.

Condição de corrida: acontece quando várias tarefas acessam/alteram o **mesmo dado ao mesmo tempo**, e o resultado pode ficar errado dependendo da ordem de execução.

13. **Mutex, Semáforos e Monitores**

Os três servem para **controlar o acesso a recursos compartilhados** e evitar problemas de concorrência, como a condição de corrida.

Mutex → UM POR VEZ

Mutex funciona como uma **chave**, apenas uma tarefa pode acessar o recurso protegido por vez.

Exemplo: duas pessoas querem alterar o mesmo saldo. Uma entra, a outra precisa esperar.

Semáforo → CONTROLA QUANTOS

O **Semáforo** controla **quantas tarefas podem acessar um recurso ao mesmo tempo**.

Exemplo: estacionamento com **3 vagas** → entram até 3 carros. Quando está cheio, os próximos esperam.

Monitor → ORGANIZA O ACESSO

O **Monitor** é uma estrutura de sincronização que **organiza/protege o acesso aos dados compartilhados**, permitindo que as tarefas esperem quando necessário.

14. **Deadlock**

Deadlock: acontece quando **dois ou mais processos ficam esperando recursos uns dos outros e nenhum consegue continuar**.

 **Macete:** todos esperando, ninguém continua.

15. **Sincronização, Lamport e Relógios Vetoriais**

Sincronização: serve para **coordenar processos e organizar a ordem dos eventos**.

Relógio de Lamport: usa números lógicos para **ajudar a ordenar os eventos** de um sistema distribuído.

Relógio Vetorial: usa um conjunto de valores para identificar melhor a **relação entre os eventos**, inclusive se são concorrentes.

16. **Coordenador, Bully e Barreiras**

Coordenador → ORGANIZA

Coordenador: processo escolhido para **coordenar/organizar tarefas** no sistema.

Algoritmo de Bully → ESCOLHE COORDENADOR

Bully: algoritmo usado para **eleger um novo coordenador**; normalmente vence o processo ativo de maior ID.

Barreira → ESPERAR TODOS

Barreira: faz os participantes **esperarem até que todos cheguem ao mesmo ponto para continuar.**

17. Tolerância a Falhas: Retry e Replicação

Retry → TENTAR DE NOVO

Se uma operação falhou, o sistema pode **tentar executá-la novamente.**

Replicação → TER CÓPIAS

mantém **cópias de dados ou serviços** para aumentar a disponibilidade e ajudar na tolerância a falhas.

18. Checkpointing, Logs e Monitoramento

Checkpointing → SALVAR O ESTADO

O sistema salva um **ponto do seu estado atual.** Se ocorrer uma falha, pode recuperar a execução a partir daquele ponto, em vez de começar tudo novamente.

Logs → REGISTRAR

Logs são **registros do que aconteceu no sistema**, como eventos, erros e operações.

Monitoramento → OBSERVAR

É acompanhar o sistema para verificar **como ele está funcionando** e identificar problemas.

19. Balanceamento de Carga e Escalabilidade

Balanceamento de carga → DIVIDIR O TRABALHO

Quando existem vários servidores, o sistema pode **distribuir as solicitações entre eles**, evitando sobrecarregar apenas um.

Escalabilidade → CRESCER

É a capacidade do sistema de **aumentar seus recursos para atender uma demanda maior**.

20. **Segurança: SSL/TLS**

SSL/TLS → PROTEGER A COMUNICAÇÃO

SSL/TLS: são tecnologias usadas para **proteger os dados durante a comunicação pela rede**, principalmente por meio de criptografia.

Exemplo curto para anotar

Quando acessa um site usando **HTTPS**, o **TLS** ajuda a proteger os dados trocados entre seu navegador e o servidor.

21. Sistemas Centralizados x Distribuídos

Sistema centralizado: as operações e o armazenamento ficam concentrados em um ponto/servidor principal.

Sistema distribuído: vários computadores se comunicam e coordenam para atingir um objetivo comum.

Macete: Centralizado = UM ponto. Distribuído = VÁRIAS máquinas trabalhando juntas.

22. Microsserviços

Microsserviços: divide a aplicação em serviços independentes, permitindo desenvolver e atualizar partes separadamente.

Vantagem: facilita manutenção e atualizações sem impactar o sistema inteiro.

Macete: Microsserviços = DIVIDIR em serviços independentes.

23. TCP/IP, IP, Latência, Perda de Pacotes, Portas e Sockets

TCP/IP: permite transmitir dados entre máquinas; o TCP organiza a entrega e o IP cuida do endereçamento dos pacotes.

IP: é responsável por endereçar os pacotes para que cheguem ao destino correto.

Latência: é o atraso no envio e recebimento dos dados.

Perda de pacotes: acontece quando dados enviados não chegam ao destino.

Portas e sockets: ajudam a identificar e direcionar a comunicação para o serviço/processo correto.

Macete: IP = ENDEREÇO. Latência = ATRASO. Perda = NÃO CHEGOU. Porta/Socket = DIRECIONAR.

24. Exclusão Mútua e Ricart-Agrawala

Exclusão mútua: garante que apenas um processo acesse uma região crítica/recurso compartilhado por vez.

Ricart-Agrawala: quando um nó quer acessar o recurso, envia uma mensagem com um timestamp (relógio lógico) aos outros nós.

Macete: Exclusão mútua = UM POR VEZ. Ricart-Agrawala = PEDIR acesso usando mensagens e tempo lógico.

25. Produtor-Consumidor e Leitores-Escritores

Produtor-Consumidor: o produtor pode tentar inserir dados em um buffer cheio e o consumidor pode tentar retirar de um buffer vazio.

Solução: usar buffer compartilhado controlado por semáforos e mutex.

Leitores-Escritores: vários leitores podem ler, mas um escritor alterando o dado durante uma leitura pode causar inconsistência.

Solução: usar mutex ou semáforo de exclusão para controlar o acesso.

Macete: Produtor/Consumidor = controlar BUFFER. Leitor/Escritor = controlar LEITURA e ALTERAÇÃO.

26. Paxos e Raft - Consenso

Paxos e Raft: algoritmos de consenso que ajudam vários computadores a concordarem com uma única decisão ou valor mesmo diante de falhas.

Consistência: ajudam os nós a manter uma visão consistente do estado do sistema.

Líder e replicação: um líder coordena decisões e os logs de estado podem ser replicados para outros nós; se o líder falhar, outro pode ser eleito.

Macete: Paxos/Raft = CONCORDAR mesmo com falhas.

27. Deadlock - Prevenção, Detecção e Recuperação

Prevenção: criar regras para evitar que o deadlock aconteça.

Detecção: usar mecanismos para descobrir ciclos de espera entre processos.

Recuperação: liberar recursos ou reiniciar/derrubar um dos processos travados.

Macete: Prevenir = EVITAR. Detectar = ENCONTRAR. Recuperar = DESTRAVAR.

28. Cache e CDN

Cache: mantém dados usados com frequência mais próximos para reduzir o tempo de acesso.

CDN: pode entregar conteúdo a partir de pontos mais próximos do usuário, reduzindo a latência.

Macete: Cache/CDN = conteúdo MAIS PERTO = menos atraso.

29. Idempotência, Rollback e Consistência

Idempotência: usa uma identificação única por tentativa para evitar que uma repetição gere duas cobranças ou duas reservas.

Rollback: desfaz uma operação pendente ou problemática para recuperar um estado consistente.

Consistência: busca impedir dados divergentes e manter o sistema confiável.

Macete: Idempotência = NÃO DUPLICAR. Rollback = DESFAZER. Consistência = MANTER CERTO.